
```
<script type="text/javascript">
root = {"data": "A", "children": [{"data": "B", "children":
[{"data": "D", "children": []}]}, {"data": "C", "children":
[{"data": "E", "children": [{"data": "G", "children": []},
{"data": "H", "children": []}]}, {"data": "F", "children": []}]}};

function postorderTraverse(node) {
    for (let i = 0; i < node["children"].length; i++) {
        // RECURSIVE CASE
        postorderTraverse(node["children"][i]); // Traverse child nodes.
    }
    document.write(node["data"] + " "); // Access this node's data.
    // BASE CASE
    return;
}

postorderTraverse(root);
</script>
```

The output of these programs is the node data in postorder order:

D B G H E F C A

The postorder traversal order of the nodes displays the data in left nodes before right nodes, and in bottom nodes before top nodes. When we compare the `postorderTraverse()` and `preorderTraverse()` functions, we find that the names are a bit of a misnomer: *pre* and *post* don't refer to the order in which nodes are visited. The nodes are always traversed in the same order; we go down the child nodes first (called a *depth-first search*) as opposed to visiting the nodes in each level before going deeper (called a *breadth-first search*). The *pre* and *post* refer to *when* the node's data is accessed: either before or after traversing the node's children.

Inorder Tree Traversal

Binary trees are tree data structures with at most two child nodes, often called the *left child* and *right child*. An *inorder tree traversal* traverses the left child node, then accesses the node's data, and then traverses the right child node. This traversal is used in algorithms that deal with binary search trees (which are beyond the scope of this book). The *inorderTraversal.py* program contains Python code that performs this kind of traversal: